

Systeme de prediction des prix de l'imoobilier

INTRODUCTION

La prediction des prix immobilier est un exercice informatique delicats et fastidieux du fait de la difficulté de récolté des données fiables mais néanmoins reste un exercice intéressant. Grâce à nos nombreuses recherches nous avons pu mettre sur pieds une méthode de prédiction de données

Cette methode ce decrit en 4 principaux point à savoir:

- Recuperation des donnée
- Normalisation des données par grâce à la normalisation de z-score
- Determination d'un indice d'impact (pour la suite du document appelé II)
- Creation de graphe et prediction

1- Recupération des données

Nous allons récupérer nos données dans un fichier excel faisant office de dataset.

```
In [17]: import pandas as pd
#toute les données de quartiers sont rassemblés dans un même fichier mais nous allos travailler avec Akwa
df=pd.read_excel('rassamble.xlsx',sheet_name='AKWA')
print("les donnée récupéré sont:")

#affichage
print(df)

#supression des colones inutiles
df=df.drop(columns='A')
print("\n les données après supression des colonnes inutiles sont: \n")
print(df)
```

les donnée récupéré sont:

	A	2000	2001	2002	2003	2004	2005	\
0	ecole	2	2	2	3	4	5	
1	hopitaux	1	3	5	6	7	10	
2	hotel	5	5	5	5	5	9	
3	pharmacie	0	2	2	2	2	2	
4	restaurant	3	5	7	17	16	20	
5	studio	20000	20000	20000	20000	20000	30000	
6	appartement 2 chambre	25000	25000	30000	30000	40000	50000	
7	appartement 3 chambres	100000	110000	110000	110000	115000	115000	
8	maison	400000	400000	400000	400000	450000	480000	

	2006	2007	2008	...	2015	2016	2017	2018	2019	\
0	7	12	12	...	20	20	20	25	29	
1	14	15	18	...	27	27	27	30	37	
2	10	13	16	...	22	27	24	24	24	
3	3	4	8	...	15	15	15	15	15	
4	20	28	40	...	75	75	75	80	80	
5	30000	30000	30000	...	40000	40000	40000	40000	40000	
6	50000	45000	50000	...	70000	100000	100000	100000	130000	
7	120000	130000	150000	...	270000	270000	285000	290000	310000	
8	490000	500000	520000	...	750000	790000	800000	840000	870000	

	2020	2021	2022	2023	2024
0	29	29	28	31	31
1	36	35	35	35	35
2	24	27	29	29	29
3	15	18	20	20	20
4	80	83	83	82	82
5	45000	45000	45000	45000	45000
6	130000	130000	140000	150000	150000
7	320000	340000	340000	340000	350000
8	890000	960000	970000	990000	1000000

[9 rows x 26 columns]

les données après supression des colonnes inutiles sont:

	2000	2001	2002	2003	2004	2005	2006	2007	2008	\
0	2	2	2	3	4	5	7	12	12	
1	1	3	5	6	7	10	14	15	18	
2	5	5	5	5	5	9	10	13	16	
3	0	2	2	2	2	2	3	4	8	
4	3	5	7	17	16	20	20	28	40	
5	20000	20000	20000	20000	20000	30000	30000	30000	30000	
6	25000	25000	30000	30000	40000	50000	50000	45000	50000	
7	100000	110000	110000	110000	115000	115000	120000	130000	150000	
8	400000	400000	400000	400000	450000	480000	490000	500000	520000	

	2009	...	2015	2016	2017	2018	2019	2020	2021	\
0	12	...	20	20	20	25	29	29	29	
1	18	...	27	27	27	30	37	36	35	
2	18	...	22	27	24	24	24	24	27	
3	10	...	15	15	15	15	15	15	18	
4	46	...	75	75	75	80	80	80	83	
5	30000	...	40000	40000	40000	40000	40000	45000	45000	
6	60000	...	70000	100000	100000	100000	130000	130000	130000	
7	160000	...	270000	270000	285000	290000	310000	320000	340000	
8	540000	...	750000	790000	800000	840000	870000	890000	960000	

	2022	2023	2024
0	28	31	31
1	35	35	35
2	29	29	29
3	20	20	20
4	83	82	82
5	45000	45000	45000
6	140000	150000	150000
7	340000	340000	350000
8	970000	990000	1000000

[9 rows x 25 columns]

2- Normalisation des donnée par la methode de normalisation de z-score

La normalisation z-score consiste à soustraire la moyenne et à la diviser par l'ecart-type pour obtenir une distribution centrée en 0 et avec une varicance de 1. Cette normalisation rend le donnée plus digest et mettent sous une forme similaire tout en maintenant quasiment toute leur caractérisque Cette normalisation est choisi car elle n'est pas tres sensible aux valeurs manquante et abhérante.

```
In [23]: import numpy as np
from scipy.stats import zscore

#Transformation des donnée prise en donnée excel
data=df.to_numpy()

#On récupère les prix des appartements a 2 chambres
prix=data[6]
print("prix des appartement a 2 chambres")
print(prix)

#supprime les prix
data=np.delete(data, (5,6,7,8),axis=0)
print("Tableau sans prix")
print(data)

#convertir le tableau en un tableau de reels afin d'avoir des donnée plus precise
data=data.astype(np.float64)

#normalisation du tableau grace a un parcourt de ligne et la methode z-score de la bibliothèque scipy
for x in range(0,data.shape[0],1):
    data[x]=zscore(data[x])

#affichage du tableau normaliser
print("tableau normaliser :")
print(data)
```

```
prix des appartement a 2 chambres
[ 25000  25000  30000  30000  40000  50000  50000  45000  50000  60000
  60000  60000  70000  75000  75000  70000 100000 100000 100000 130000
 130000 130000 140000 150000 150000]
Tableau sans prix
[[ 2  2  2  3  4  5  7 12 12 12 12 15 17 19 20 20 20 20 25 29 29 29 28 31
 31]
 [ 1  3  5  6  7 10 14 15 18 18 17 20 20 25 28 27 27 27 30 37 36 35 35 35
 35]
 [ 5  5  5  5  5  9 10 13 16 18 19 20 20 22 22 22 27 24 24 24 24 27 29 29
 29]
 [ 0  2  2  2  2  2  3  4  8 10 14 14 14 14 17 15 15 15 15 15 15 18 20 20
 20]
 [ 3  5  7 17 16 20 20 28 40 46 53 60 62 60 75 75 75 75 80 80 80 83 83 82
 82]]
tableau normaliser :
[[-1.44658089 -1.44658089 -1.44658089 -1.34499515 -1.24340941 -1.14182368
 -0.9386522 -0.43072352 -0.43072352 -0.43072352 -0.43072352 -0.12596631
  0.07720516  0.28037663  0.38196237  0.38196237  0.38196237  0.38196237
  0.88989105  1.296234  1.296234  1.296234  1.19464826  1.49940547
  1.49940547]
 [-1.80010771 -1.62223146 -1.4443552 -1.35541707 -1.26647894 -0.99966456
 -0.64391205 -0.55497392 -0.28815954 -0.28815954 -0.37709766 -0.11028328
 -0.11028328  0.33440736  0.60122175  0.51228362  0.51228362  0.51228362
  0.779098  1.4016649  1.31272677  1.22378864  1.22378864  1.22378864
  1.22378864]
 [-1.57054696 -1.57054696 -1.57054696 -1.57054696 -1.57054696 -1.09172167
 -0.97201534 -0.61289637 -0.25377741 -0.01436476  0.10534156  0.22504789
  0.22504789  0.46446053  0.46446053  0.46446053  1.06299215  0.70387318
  0.70387318  0.70387318  0.70387318  1.06299215  1.3024048  1.3024048
  1.3024048 ]
 [-1.65760534 -1.35731452 -1.35731452 -1.35731452 -1.35731452 -1.35731452
 -1.20716911 -1.05702369 -0.45644205 -0.15615123  0.44443042  0.44443042
  0.44443042  0.44443042  0.89486665  0.59457583  0.59457583  0.59457583
  0.59457583  0.59457583  0.59457583  1.04501206  1.34530288  1.34530288
  1.34530288]
 [-1.7261167 -1.65606326 -1.58600983 -1.23574264 -1.27076936 -1.13066248
 -1.13066248 -0.85044873 -0.43012811 -0.21996779  0.02521924  0.27040627
  0.34045971  0.27040627  0.79580705  0.79580705  0.79580705  0.79580705
  0.97094064  0.97094064  0.97094064  1.0760208  1.0760208  1.04099408
  1.04099408]]
```

3- Determination d'un indice d'impact

L'indice d'impact représente la moyenne pondéré des différentes valeurs des critère de classification sur une même année. Pour notre methode nous allons prendre la même pondération pour chaque critère pour les 50 premiers pourcent. Pour les 50derniers pourcent nous allons les attribuer à l'inflation de cette manière on se rapproche un peu plus de la réalité car les batiments peuvent rester la même au cours du temps mais l'inflation elle est difficilement stable et influence plus le prix des batiments

On calcule cet indice afin de créer un lien intime et logique entre les données

```
In [34]: #creation du tableau pour le stockage des données
```

```
#pour cet exemple on doit faire une regression linéaire simple d'où on ne travailleras qu'avec l' II et le prix

#creation du tableau pour sauvegarder les différents II de chaque année
saveTab=[]

#On multiplie directement par 0.1 vu que c'est la même pondération
dataPondere=data*0.1

# creation du tableau du taux d'inflation pour chaque année
inflation=[2.5,2.7,2.9,3.1,3.3,3,2.8,2.6,2.4,2.2,2.8,2.7,2.6,2.5,2.4,2.9,2.3,2.2,2.1,2.0,2.3,2.4,3.3,3.6,4.0]

print("tableau obtenue après multiplication par les pondération")
print(dataPondere)

#remplissage des II
for x in range(0,25,1):
    saveTab.append(np.sum(dataPondere[:,x])+(inflation[x]*0.5))
print(saveTab)

# creation du tableau du prix par rapport a l'II
dataPrix=np.array((saveTab,prix))

#creation du tableau de l'II par rapport a l'année
dataAnnee=np.array((saveTab,[2000, 2001, 2002, 2003, 2004, 2005, 2006, 2007, 2008, 2009, 2010, 2011, 2012, 2013
```

```
tableau obtenue après multiplication par les pondération
[[-0.14465809 -0.14465809 -0.14465809 -0.13449952 -0.12434094 -0.11418237
  -0.09386522 -0.04307235 -0.04307235 -0.04307235 -0.04307235 -0.01259663
  0.00772052  0.02803766  0.03819624  0.03819624  0.03819624  0.03819624
  0.08898911  0.1296234  0.1296234  0.1296234  0.11946483  0.14994055
  0.14994055]
 [-0.18001077 -0.16222315 -0.14443552 -0.13554171 -0.12664789 -0.09996646
  -0.0643912  -0.05549739 -0.02881595 -0.02881595 -0.03770977 -0.01102833
  -0.01102833  0.03344074  0.06012217  0.06012217  0.05122836  0.05122836
  0.0779098  0.14016649  0.13127268  0.12237886  0.12237886  0.12237886
  0.12237886]
 [-0.1570547  -0.1570547  -0.1570547  -0.1570547  -0.1570547  -0.10917217
  -0.09720153 -0.06128964 -0.02537774 -0.00143648  0.01053416  0.02250479
  0.02250479  0.04644605  0.04644605  0.04644605  0.10629922  0.07038732
  0.07038732  0.07038732  0.10629922  0.13024048  0.13024048
  0.13024048]
 [-0.16576053 -0.13573145 -0.13573145 -0.13573145 -0.13573145 -0.13573145
  -0.12071691 -0.10570237 -0.0456442  -0.01561512  0.04444304  0.04444304
  0.04444304  0.04444304  0.08948667  0.05945758  0.05945758  0.05945758
  0.05945758  0.05945758  0.05945758  0.10450121  0.13453029  0.13453029
  0.13453029]
 [-0.17261167 -0.16560633 -0.15860098 -0.12357426 -0.12707694 -0.11306625
  -0.11306625 -0.08504487 -0.04301281 -0.02199678  0.00252192  0.02704063
  0.03404597  0.02704063  0.07958071  0.07958071  0.07958071  0.07958071
  0.09709406  0.09709406  0.09709406  0.10760208  0.10760208  0.10409941
  0.10409941]
 [0.42990423995401394, 0.5847262915663085, 0.7095192609443794, 0.8635983661639157, 0.9791480807427492, 0.92788130
 96106922, 0.9107588815839811, 0.9493933756997033, 1.014076938084421, 0.9890633162180873, 1.3767170033293574, 1.4
 203634981505724, 1.3976859891887337, 1.4294081214251029, 1.5138318350033275, 1.724908939956496, 1.48476210155869
 07, 1.398850204597374, 1.4438378706306012, 1.49672885488947, 1.6378350420768617, 1.7704047652061883, 2.264216538
 438636, 2.441189587490168, 2.6411895874901683]
```

4- Creation de graphe et prediction

Afin nous arrivons au terme de notre maniement des donnée ici nous allons simplement utilisé les bibliothèque python afin de réalisé le modèle de prediction. Pour notre modèle nous ne ferons pas de la regression linéaire multiple mais nous ferons 2 regressions linéaires simple que nous allons lié

```
In [35]: import matplotlib.pyplot as plt1
import matplotlib.pyplot as plt2

from sklearn.linear_model import LinearRegression

#X1: variable des II, Y1: variable des prix
X1=np.array(dataPrix[0])
X1 = X1.reshape(-1, 1) #Pour eviter des erreurs sur la dimension du tableau
Y1=np.array(dataPrix[1])

#X2: variable des années, Y2: variable des II
X2=np.array(dataAnnee[1])
X2 = X2.reshape(-1, 1) #Pour eviter des erreurs sur la dimension du tableau
Y2=np.array(dataAnnee[0])

# Création des modèles de régression linéaire
model1 = LinearRegression()
model2 = LinearRegression()
```

```

# Entraîner les modèles
model1.fit(X1, Y1)
model2.fit(X2, Y2)

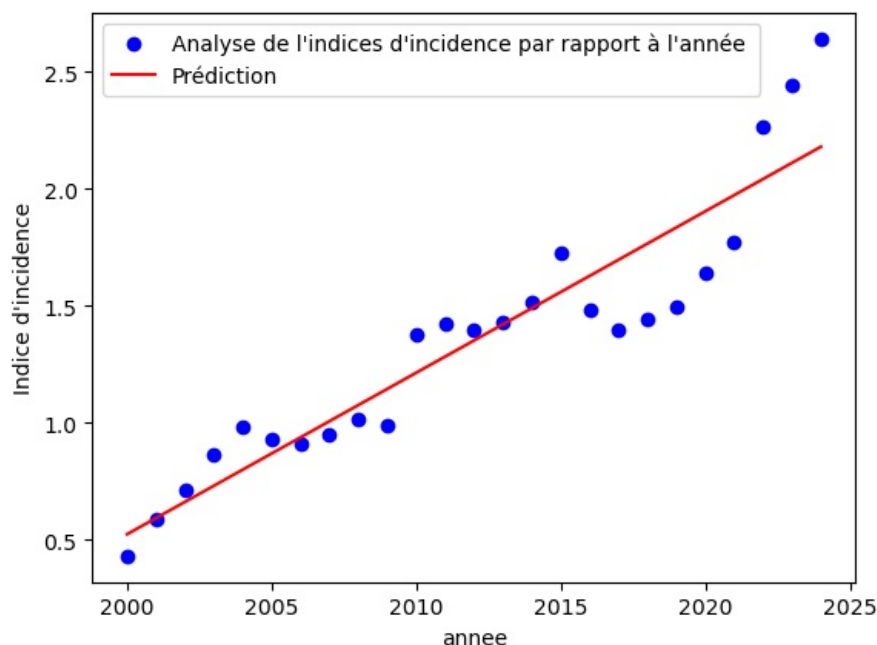
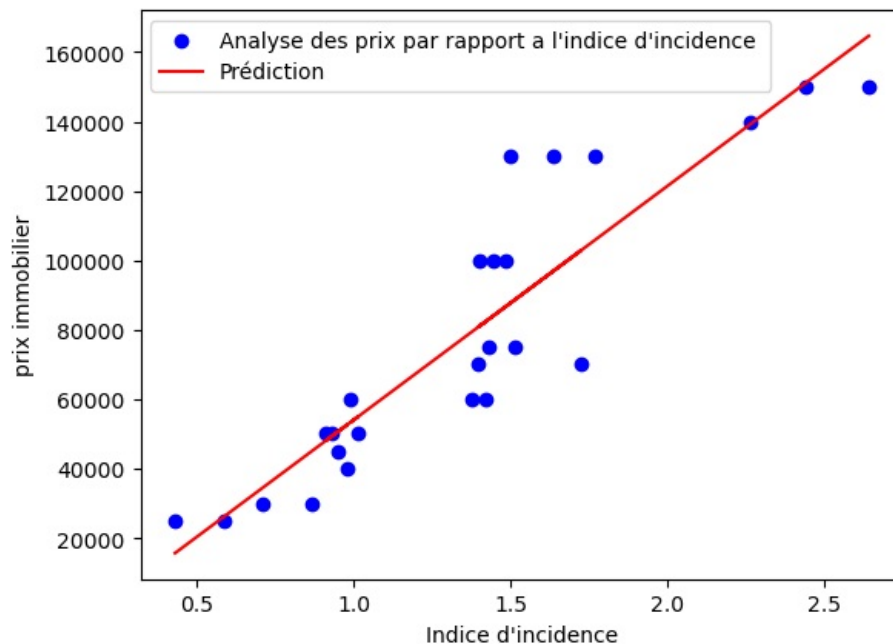
# Prédire les valeurs
Y_pred1 = model1.predict(X1)
Y_pred2 = model2.predict(X2)

# Visualiser les données des prix par rapport à l'indice d'incidence
plt1.scatter(X1, Y1, color='blue', label="Analyse des prix par rapport a l'indice d'incidence ")
plt1.plot(X1, Y_pred1, color='red', label='Prédiction')
plt1.xlabel("Indice d'incidence")
plt1.ylabel('prix immobilier')
plt1.legend()
plt1.show()

#visualisation des donnée de l'indices d'incidence par rapport à l'année
plt2.scatter(X2, Y2, color='blue', label="Analyse de l'indices d'incidence par rapport à l'année ")
plt2.plot(X2, Y_pred2, color='red', label='Prédiction')
plt2.xlabel("annee")
plt2.ylabel("Indice d'incidence")
plt2.legend()
plt2.show()

#Calcul du prix pour l'année 2030
annee=2030
ii_annee= (model2.coef_[0])*annee + model2.intercept_ #c'est a dire Y2=aX2+b
prix_annee=(model1.coef_[0])*ii_annee + model1.intercept_ #c'est a dire Y1=aX1+b
print(ii_annee)
print(f"En 2030 le prix d'un appartement à 2 chambres sera: {prix_annee}")

```



2.594491388785258

En 2030 le prix d'un appartement à 2 chambres sera: 161535.09444890666

CONCLUSION

En somme cette méthode permet de créer un coérhence entre les données rassemblés ce qui permet de creer un lien entre l'environnement du bien immobilier, ses caracteristiques propres et son prix.

Loading [MathJax]/jax/output/CommonHTML/fonts/TeX/fontdata.js